

Infinite Aura - Living Hydration Template

Last Updated: [Date - UPDATE THIS]

Current Phase: Phase 0 - Setup

Status: Not Started

Total Time Invested: 0 hours



QUICK STATUS (Update After Each Phase)

✓ Completed:

- [] Phase 0: Setup (Docker, TypeScript, Daniel's repo)
- [] Phase 1: Memory Scaffold (18 dirs, 20+ tests, hardened)
- [] Phase 2: Capture Hooks (4 hooks, session logging)
- [] Phase 3: CLI Interface (query, search, context loading)
- [] Phase 4: Orchestrator (sub-agent spawning, task routing)



In Progress:

- None yet



Next Up:

- Phase 0: Setup (Docker dev container, TypeScript project, Daniel's repo)
-



CURRENT LOCATION

Project Repo: /workspaces/infinite-aura-ts/ (or C:\dev\infinite-aura-ts)

Memory Repo: ~/.infinite-aura/memory/

Git Status: Not initialized yet

Last Commit: None yet



3 LENSES INSIGHTS (Update After Each Phase)

Lens A: Architecture (What We've Learned)

Phase 0 Insights:

- [Add after Phase 0 complete]

Phase 1 Insights:

- [Add after Phase 1 complete]

Phase 2 Insights:

- [Add after Phase 2 complete]

Phase 3 Insights:

- [Add after Phase 3 complete]

Phase 4 Insights:

- [Add after Phase 4 complete]

Lens B: Execution (How We Build)**Phase 0 Insights:**

- [Add after Phase 0 complete]

Phase 1 Insights:

- [Add after Phase 1 complete]

Phase 2 Insights:

- [Add after Phase 2 complete]

Phase 3 Insights:

- [Add after Phase 3 complete]

Phase 4 Insights:

- [Add after Phase 4 complete]

Lens C: Dynamic (How We Work)**Phase 0 Insights:**

- [Add after Phase 0 complete]

Phase 1 Insights:

- [Add after Phase 1 complete]

Phase 2 Insights:

- [Add after Phase 2 complete]

Phase 3 Insights:

- [Add after Phase 3 complete]

Phase 4 Insights:

- [Add after Phase 4 complete]

**PHASE SUMMARIES (Update After Each Phase)****Phase 0: Setup**

Status: 🕒 Not Started

Duration: [X hours - UPDATE WHEN COMPLETE]

Started: [Date]

Completed: [Date]

Key Deliverables:

- [] Docker dev container configured (.devcontainer/)
- [] TypeScript project initialized (package.json, tsconfig.json)

- [] Daniel's repo cloned (/workspaces/daniel-pai/)
- [] Project structure created (src/, tests/, docs/)

Lessons Learned:

- [Add after completion]

Quality Metrics:

- Setup time: [X hours]
- Issues encountered: [List]
- Blockers: [List]

Code Locations:

- Project: /workspaces/infinite-aura-ts/
 - Daniel's repo: /workspaces/daniel-pai/
-

Phase 1: Memory Scaffold

Status: 🕒 Not Started

Duration: [X hours - UPDATE WHEN COMPLETE]

Started: [Date]

Completed: [Date]

Key Deliverables:

- [] 18 directories created (max 3 levels)
- [] 5 key markdown files with YAML frontmatter
- [] Exception classes (6 types)
- [] MemoryScaffold class
- [] 20+ tests (100% passing)
- [] 85%+ code coverage
- [] Security hardened (path traversal, symlinks, permissions)
- [] Documentation complete (README, STATUS, CLAUDE)

Lessons Learned:

- [Add after completion]

Quality Metrics:

- Tests: [X passing / Y total]
- Coverage: [Y%]
- Security: [Pass/Fail]
- TypeScript strict: [Pass/Fail]
- ESLint: [Pass/Fail]
- Prettier: [Pass/Fail]

Code Locations:

- Scaffold: src/memory/scaffold.ts
 - Exceptions: src/memory/exceptions.ts
 - Tests: tests/memory/scaffold.test.ts
 - Integration tests: tests/memory/integration.test.ts
-

Phase 2: Capture Hooks

Status: 🕒 Not Started

Duration: [X hours - UPDATE WHEN COMPLETE]

Started: [Date]

Completed: [Date]

Key Deliverables:

- [] 4 core hooks implemented (Stop, SubagentStop, SessionStart, SessionEnd)
- [] Session logging working
- [] Content-based routing
- [] Agent-based routing
- [] Raw events in JSONL format
- [] Tests for all hooks
- [] Integration with Phase 1 scaffold

Lessons Learned:

- [Add after completion]

Quality Metrics:

- Tests: [X passing / Y total]
- Coverage: [Y%]
- Hook performance: [X ms overhead]

Code Locations:

- [Add after completion]
-

Phase 3: CLI Interface

Status: 🕒 Not Started

Duration: [X hours - UPDATE WHEN COMPLETE]

Started: [Date]

Completed: [Date]

Key Deliverables:

- [] Query command (search memory)
- [] Search command (full-text search)
- [] Context loading command
- [] CLI tests
- [] Documentation

Lessons Learned:

- [Add after completion]

Quality Metrics:

- [Add after completion]

Code Locations:

- [Add after completion]
-

Phase 4: Orchestrator

Status: 🕒 Not Started

Duration: [X hours - UPDATE WHEN COMPLETE]

Started: [Date]

Completed: [Date]

Key Deliverables:

- [] Orchestrator agent (single point of contact)
- [] Sub-agent spawning
- [] Task routing
- [] Result validation
- [] Context loading from memory
- [] Integration tests
- [] End-to-end tests

Lessons Learned:

- [Add after completion]

Quality Metrics:

- [Add after completion]

Code Locations:

- [Add after completion]



KEY DECISIONS LOG (Add After Each Decision)

Decision 1: Switch to TypeScript

- **When:** December 31, 2024
- **Why:** Daniel's 2 years optimization, MIT licensed, direct code access
- **Impact:** Cleaner implementation, better tooling, proven patterns
- **Alternatives Considered:** Stay with Python
- **Outcome:** Proceeding with TypeScript

Decision 2: [Next decision]

- **When:** [Date]
- **Why:** [Reason]
- **Impact:** [Impact]
- **Alternatives Considered:** [List]
- **Outcome:** [Result]

[Continue adding decisions as they're made]



TECHNICAL CONTEXT (Update As We Build)

Current Stack

- **Language:** TypeScript

- **Runtime:** Node.js 20 LTS
- **Testing:** Jest
- **Quality:** ESLint, Prettier, TypeScript strict mode
- **Container:** Docker dev container
- **Package Manager:** npm

Current Structure

```
/workspaces/infinite-aura-ts/
├── .devcontainer/
│   ├── devcontainer.json
│   └── Dockerfile
├── .vscode/
├── src/
│   ├── memory/
│   │   ├── scaffold.ts
│   │   ├── exceptions.ts
│   │   └── index.ts
│   └── index.ts
├── tests/
│   ├── memory/
│   │   ├── scaffold.test.ts
│   │   └── integration.test.ts
├── docs/
│   ├── README.md
│   ├── STATUS.md
│   └── CLAUDE.md
├── package.json
├── tsconfig.json
└── jest.config.js
```

```
~/.infinite-aura/memory/
├── history/
├── learnings/
├── decisions/
├── projects/
├── agents/
├── sessions/
├── skills/
├── index/
├── backups/
├── meta/
│   ├── templates/
│   ├── guardrails/
│   └── version.md
├── raw-events/
│   └── README.md
├── user_context.md
├── current_projects.md
└── orchestrator_state.md
```

Dependencies

- [Add as we install packages]

Environment

- **OS:** Windows 11 with WSL2
- **Docker:** Docker Desktop

- **VS Code:** With Remote-Containers extension
 - **Git:** Initialized in both repos
-



WHAT TO REFERENCE (Update As We Go)

For Phase 0 (Setup)

- Handoff document Part 9 (PROMPTS 1-3)
- Docker documentation
- TypeScript documentation
- Node.js documentation

For Phase 1 (Memory Scaffold)

- Handoff document Part 9 (PROMPTS 4-7)
- Daniel's repo: /workspaces/daniel-pai/
- 1_pai_kai_memory_system_analysis.md (80 pages)
- Python Phase 1 code (reference only, not copying)

For Phase 2 (Hooks)

- Daniel's hook system (in his repo)
- Phase 1 code (scaffold)
- Hook system documentation
- [Add more as we go]

For Phase 3 (CLI)

- Daniel's CLI patterns
- Phase 1 & 2 code
- [Add when we get there]

For Phase 4 (Orchestrator)

- All previous phases
 - Multi-agent patterns
 - [Add when we get there]
-



BLOCKERS & ISSUES (Track As They Arise)

Current Blockers

- None yet

Resolved Issues

- [Add as we resolve issues]

Questions for Daniel

Status: Prepared, waiting for call

Critical Questions:

1. Hook event ordering - sync or async?
2. Content routing logic - priority order?
3. Agent collision handling - registry pattern?
4. Hook failure impact - retry or skip?

Important Questions:

5. Hook performance - milliseconds overhead?
6. Raw events use cases - analytics or debugging?
7. Observability dashboard - architecture?
8. Skill routing - lazy-load or index?

Exploratory Questions:

9. Voice system - edge cases?
10. Packs dependencies - version conflicts?
11. Self-improvement - Principle #10 in practice?
12. Multi-user - team scenarios?

Answers:

- [Add after Daniel's call]

**METRICS & PROGRESS****Time Tracking**

- **Phase 0:** [X hours]
- **Phase 1:** [X hours]
- **Phase 2:** [X hours]
- **Phase 3:** [X hours]
- **Phase 4:** [X hours]
- **Total:** 0 hours (update as we go)

Quality Metrics

- **Total tests:** 0 (target: 20+ per phase)
- **Total coverage:** 0% (target: 85%+)
- **Security issues:** 0 (target: 0)
- **Code quality:** Not started (target: All checks passing)

Progress

- **Overall completion:** 0%
- **Current phase completion:** 0%

**IF WE RESET (How to Resume)****What to Upload to New Chat**

1. **This Living Hydration Template** (shows current status)
2. **Handoff document** (typescript_migration_handoff.md - full context)

3.  **1_pai_kai_memory_system_analysis.md** (80 pages - Daniel's architecture)
4.  **Key .md docs** (if needed for specific context)

What to Say

I'm continuing Infinite Aura TypeScript build.

Current status: Phase [X], [Y%] complete

I have:

1. Living Hydration Template (current progress, all context)
2. Comprehensive handoff document (full project context)
3. 80-page PAI/KAI analysis
4. Supporting documentation

Please review the Living Hydration Template FIRST to see where we are, then review the handoff document for full context.

We're **at** [specific point in Phase X]. Ready **to continue from** here.

[Paste any specific context about current work]

The New Agent Will

1. Read Living Template (understand current status)
2. Read Handoff (understand full context)
3. Review supporting docs (as needed)
4. Continue from where you left off
5. Update Living Template as work progresses



NOTES & OBSERVATIONS (Free-form)

General Notes

- [Add any observations, ideas, or notes as we go]

Things to Remember

- Memory-first philosophy (build memory before orchestration)
- Two-repo pattern (project + memory)
- Quality standards (85%+ coverage, security hardened)
- Daniel's patterns (hooks, history, UFC)

Future Improvements

- [Ideas for later phases or enhancements]



UPDATE CHECKLIST (Use This After Each Phase)

After EVERY phase completion:

- [] Update "Last Updated" date at top
- [] Update "Current Phase"

- [] Update "Status" (In Progress / Complete)
- [] Check off completed deliverables for that phase
- [] Add "Lessons Learned" for that phase
- [] Add "Quality Metrics" for that phase
- [] Update "3 Lenses Insights" with new learnings
- [] Add any new decisions to "Key Decisions Log"
- [] Update "Time Tracking" with actual hours
- [] Update "Progress" percentages
- [] Add any blockers or issues encountered
- [] Update "Code Locations" with new files
- [] Save updated version
- [] Commit to git (if appropriate)

How to Update in Chat:

Say: "Please update the Living Hydration Template with Phase X completion details" and provide:

- What was completed
- Time taken
- Quality metrics (tests, coverage, etc.)
- Lessons learned
- Any issues or blockers
- Key insights from all 3 lenses

END OF LIVING HYDRATION TEMPLATE